

Prism: A Minimal Compositional Metalanguage for Specifying Agent Behaviour

Vanja Kljajević, PhD
University of Oslo

Franck Binard, PhD
Deloitte

Abstract

This paper introduces *Prism*, a small, compositional metalanguage for describing the behaviour of software agents that interact with their environment through external tools (for example, APIs, sensors, or actuators). The design is deliberately influenced by linguistic notions: Prism has a tiny, fixed “core grammar” and allows users to define domain-specific “lexicons” (contexts) that extend this core with new categories and predicates.

Instead of hard-wired control structures such as `if` and `else`, Prism uses ordinary expressions and an abstraction operator `|` to encode decisions as selections among alternatives. This makes agent policies look much closer to declarative, grammar-like specifications than to imperative code, while still being precise enough to serve as an executable contract for agent behaviour.

We motivate the need for such a language from the perspective of linguistics and agent design, present the structure of the core context (`Core1`), and illustrate how domain-specific contexts can be used to specify realistic agent behaviours in a fully compositional way.

1 Motivation: Agents as Situated Language Users

Large language models increasingly act as *agents*: they not only produce text but also call tools, query databases, and control physical or virtual devices. From a linguistic point of view, this raises a familiar problem in a new guise: how do we connect *utterances* (e.g. user prompts) to *actions* in a way that is transparent, compositional, and inspectable?

Two pressures pull in opposite directions:

- From engineering, the need for precise, machine-readable specifications of what an agent is allowed to do in response to different kinds of input (for safety, verification, debugging, and explanation).
- From linguistics, an interest in systems that respect compositional structure, lexicon/grammar separation, and that make the link between meaning and action explicit rather than opaque.

Prism is a proposal aimed at the overlap of these concerns. It is not a general-purpose programming language; rather, it is a compact *metalanguage* for writing down agent behaviour in a way that looks more like a grammar than like a program:

- It has a fixed, minimal core (a “background grammar”).
- Domains add their own contexts (mini-grammars) with categories such as `Location`, `Temperature`, or `Action`.

- Decisions are expressed as ordinary expressions that *select* between alternatives, rather than as special control constructs.
- External tools are treated as named capabilities with associated schemas, which can be connected directly to protocols such as the Model Context Protocol (MCP) [Anthropic, 2024], but this complexity is hidden behind a simple surface form.

The rest of the paper describes this design, focusing on how it supports agent behaviour in a way that should be familiar and interesting to linguists.

2 Abstraction and the Core Context Core1

Prism has one main syntactic operator: the vertical bar `|`, which we use as a generic *abstraction operator*. Informally, “`x | body`” can be read as “an expression that, given `x`, behaves as `body`”. We also allow multiple binders, e.g. `a, b | body`. This approach draws inspiration from Church-style abstraction in type theory [Taylor, 1999].

The core of Prism is captured as a context called **Core1**. Other contexts “extend” this core, in the same way that a specific lexicon might extend a more general one. In what follows, we show **Core1** using a concrete notation but describe it informally, without dwelling on type-theoretic detail.

2.1 Base Categories and Encoded Structures

```
----context
context Core1

Number
String
Unit
Schema
JSON
None
UserPrompt
```

These are the basic categories:

- **Number**, **String**: numerical and textual values.
- **Unit**: measurement units (e.g. `celsius`, `meters`).
- **Schema**, **JSON**: structures used to describe and represent tool inputs and outputs.
- **None**: an empty category used when a function takes no meaningful input.
- **UserPrompt**: user utterances or queries.

On top of these, **Core1** defines several structures using `|` as an abstraction mechanism.

Booleans.

```
Bool := X | X - X - X
```

Here, `Bool` is described as an abstraction over some result category `X`. The idea is: a boolean knows how to choose between two candidates of the same category. Two particular choices are singled out:

```
true  := X | t, f | t
false := X | t, f | f
```

and from these we can define the usual boolean operations as pure combinations:

```
and := b1, b2 | b1[Bool] b2 false
or  := b1, b2 | b1[Bool] true b2
not := b | b[Bool] false true
```

The bracket notation `b[Bool]` means: “interpret `b` as a choice-maker at the category `Bool`”.

Boolean Operations Example

```
-- Combining two predicates
isWarm := gtTemp currentTemp (temp 20 celsius)
isSunny := eq weatherCondition "sunny"
isNice := and isWarm isSunny
```

Lists and Predicates.

```
List T := R | (T - R - R) - R - R
Predicate := X | X - X - Bool
```

`List T` encodes the idea of a list over some category `T`—it is not used extensively in the examples here, but forms part of the general core.

`Predicate` describes binary predicates: for a given category `X`, a `Predicate[X]` is something that, given two instances of `X`, yields a `Bool`. For numbers, `Core1` declares standard comparison predicates as *external* (i.e. implemented outside the language):

```
external gt : Predicate[Number]
external lt : Predicate[Number]
external eq : Predicate[Number]
external gte : Predicate[Number]
external lte : Predicate[Number]
```

Tools and Pairs.

```
Tool := X | (String - Schema - Schema - X) - X
Pair U V := X | (U - V - X) - X
```

A `Tool` is a triple consisting of a name and input/output schemas. Intuitively, it is “a capability the agent can use”, such as a web API, a database query, or a hardware sensor.

`Pair U V` encodes a pair of a `U` and a `V`, again described in terms of how it interacts with a consumer. The core introduces a pure constructor:

```
pair := A, B | a, b | X | x | x a b
```

which can be understood as: given **a** and **b**, produce a pair that, when given a consumer **x**, applies **x** to **a** and **b**.

2.2 Core as a Shared Background Grammar

From a linguistic perspective, **Core1** supplies a small “background grammar”:

- Categories such as **Number**, **String**, **Unit**, and **UserPrompt**.
- Basic semantic operators: conjunction, disjunction, negation.
- Predicates on numbers: greater-than, less-than, equality, and their inclusive variants.
- A way to talk about tools (external capabilities).
- A way to construct simple product-like structures (pairs).

Prism itself does not have keywords like “if” or “otherwise”. Instead, such notions are encoded via these small combinators.

3 Contexts as Domain-Specific Mini-Grammars

A *context* extends **Core1** with new categories and external bindings. Syntactically it has the form:

```
----context
context Name extends Core1

... new categories ...
... definitions (:=) ...
... external tools and actions ...
```

From a linguistic perspective, a context is a small domain-specific grammar: it names entities, introduces roles (such as **Action** or **Location**), and declares which external tools are available in that domain. However, every context shares the same underlying combinatorial machinery from **Core1**.

4 Example 1: Thermostat Control

4.1 Setting up the Domain

We begin by defining a context **ThermostatControl** that extends **Core1**.

```
----context
context ThermostatControl extends Core1

type Location
type Action extends Tool
```

Location is a new category for places in the environment (e.g. **office**, **kitchen**); **Action** is a role refining **Tool**, used specifically for things the agent can do to the thermostat.

We represent temperatures as number–unit pairs:

```
Temperature := Pair[Number] [Unit]
```

Using the core `pair` constructor, we introduce a convenience function `temp` that builds a temperature from a number and a unit:

```
temp := n, u | pair[Number][Unit] n u
```

For example, `temp 23 celsius` is the temperature “23 degrees Celsius”.

We also introduce specific locations and units:

```
office  : Location
kitchen : Location
bedroom : Location
celsius : Unit
fahrenheit : Unit
```

Next, an external sensor:

```
external tempSensor : Location - Unit - Temperature
```

This is the agent’s way of reading the temperature. From the perspective of Prism, `tempSensor` behaves as something that, given a location and a unit, yields a temperature value.

Finally, we define external comparisons over temperatures, and the thermostat actions themselves:

```
external gtTemp : Predicate[Temperature]
external ltTemp : Predicate[Temperature]

external lowerThermostat : Action
external raiseThermostat : Action
maintainThermostat      : Action
```

`gtTemp` and `ltTemp` are domain-specific predicates that say when one temperature is greater or less than another. `lowerThermostat`, `raiseThermostat`, and `maintainThermostat` are actions the agent can take.

4.2 Behavioural Policy as a Prism Expression

We now express the following policy purely in Prism:

If the office temperature is above 23 °C, lower the thermostat. Otherwise, if it is below 20.5 °C, raise the thermostat. Otherwise, keep the thermostat as it is.

In Prism:

```
(gtTemp (tempSensor office celsius) (temp 23 celsius))[Action]
  lowerThermostat
((ltTemp (tempSensor office celsius) (temp 20.5 celsius))[Action]
  raiseThermostat
  maintainThermostat)
```

Read from the top:

1. `tempSensor office celsius` asks the external sensor for the temperature in the office, in Celsius.

2. `gtTemp (tempSensor office celsius) (temp 23 celsius)` is a boolean phrase: “the office temperature is greater than 23 °C”.
3. Applying this boolean to `[Action]` and two candidate actions:

```
(gtTemp (...))[Action] lowerThermostat ...
```

yields an `Action`:

- `lowerThermostat` if the condition is true;
- the “else” branch if it is false.

4. The “else” branch is itself another boolean selection:

```
(ltTemp (tempSensor office celsius) (temp 20.5 celsius))[Action]
  raiseThermostat
  maintainThermostat
```

which chooses between `raiseThermostat` and `maintainThermostat`, depending on whether the temperature is below 20.5 °C.

Crucially, nowhere do we introduce special syntax for `if`, `then`, or `else`. What looks like an “if-elseif-else” structure emerges from the interaction of three components:

- domain predicates (`gtTemp`, `ltTemp`),
- core booleans (`Bool`, `true`, `false`, and the way they act as selectors),
- and a convention of applying booleans with a target category in brackets (here `[Action]`) followed by the two candidate results.

From a linguistic perspective, the policy above is a kind of *compositionally specified decision procedure*. Each part is locally interpretable (e.g. “*the office temperature is greater than 23 °C*”) and combined into a global action choice in a way that respects a clear scopal structure.

5 Example 2: Smart Home Security

5.1 Domain Context

```
----context
context HomeSecurity extends Core1

type Room
type SecurityAction extends Tool
type SecurityLevel

living_room : Room
front_door  : Room
garage      : Room

high : SecurityLevel
medium : SecurityLevel
```

```

low      : SecurityLevel

external motionSensor : Room - Bool
external doorSensor   : Room - Bool
external alertSecurity : SecurityLevel - SecurityAction
external logEvent      : String - SecurityAction
external doNothing     : SecurityAction

```

5.2 Security Policy

Security Response Policy

“If motion is detected in the living room AND the front door is open, send a high-level security alert. Otherwise, if only motion is detected, log the event. Otherwise, do nothing.”

```

policy :=
  (and
    (motionSensor living_room)
    (doorSensor front_door))[SecurityAction]
  (alertSecurity high)
  ((motionSensor living_room)[SecurityAction]
    (logEvent "motion_detected")
    doNothing)

```

6 Example 3: E-Commerce Recommendation

6.1 Domain Context

```

----context
context ECommerce extends Core1

type Customer
type Product
type Recommendation extends Tool

external customerBudget : Customer - Number
external productPrice   : Product - Number
external customerHistory : Customer - List[Product]
external inStock         : Product - Bool

external recommend      : Product - Recommendation
external suggestAlt     : Product - Recommendation
external notifyOutOfStock : Recommendation

```

6.2 Recommendation Policy

Product Recommendation Logic

“If the product is in stock AND the price is within the customer’s budget, recommend it. Otherwise, if the product is out of stock, notify the customer. Otherwise, suggest an alternative.”

```
recommendProduct := customer, product |  
  (and  
    (inStock product)  
    (lte (productPrice product) (customerBudget customer))  
  ) [Recommendation]  
  (recommend product)  
  ((inStock product) [Recommendation]  
    (suggestAlt product)  
    notifyOutOfStock)
```

7 Example 4: Medical Alert System

7.1 Domain Context

```
----context  
context MedicalAlert extends Core1  
  
type Patient  
type VitalSign := Pair[Number][Unit]  
type AlertLevel  
type Response extends Tool  
  
bpm      : Unit -- beats per minute  
mmHg     : Unit -- millimeters of mercury  
mgdL     : Unit -- milligrams per deciliter  
  
critical : AlertLevel  
warning  : AlertLevel  
normal   : AlertLevel  
  
external heartRate      : Patient - VitalSign  
external bloodPressure : Patient - VitalSign  
external bloodSugar     : Patient - VitalSign  
  
external emergencyCall : Patient - AlertLevel - Response  
external notifyNurse   : Patient - Response  
external logVitals     : Patient - Response  
  
vitalSign := n, u | pair[Number][Unit] n u
```


7.2 Alert Policy

Patient Monitoring Policy

“If heart rate exceeds 120 bpm OR blood pressure systolic exceeds 180 mmHg, trigger emergency call. Otherwise, if heart rate exceeds 100 bpm, notify nurse. Otherwise, just log the vitals.”

```
monitorPatient := patient |  
  (or  
    (gtTemp (heartRate patient) (vitalSign 120 bpm))  
    (gtTemp (bloodPressure patient) (vitalSign 180 mmHg))  
  ) [Response]  
  (emergencyCall patient critical)  
  ((gtTemp (heartRate patient) (vitalSign 100 bpm)) [Response]  
    (notifyNurse patient)  
    (logVitals patient))
```

8 Linguistic Perspective

8.1 Compositionality and Selection

Prism is explicitly compositional: each expression is built from smaller expressions by function application and abstraction. This aligns well with the Fregean principle of compositionality, that the meaning of the whole is a function of the meanings of the parts and their mode of combination [Frege, 1892, Partee, 1995].

In the thermostat example, we can identify sub-phrases such as:

- `tempSensor office celsius`: a kind of “measurement phrase”.
- `temp 23 celsius`: a “degree phrase”.
- `gtTemp (tempSensor office celsius) (temp 23 celsius)`: a sentential predicate about the state of the world.

Instead of mapping directly to truth conditions, however, Prism maps such predicates to *choices over actions*. Booleans, when instantiated at a category like **Action**, become selection functions:

Given two alternative actions, choose one.

This is reminiscent of work in dynamic semantics and decision theory, where propositions can be seen as tests or update functions, rather than as static truth values [Groenendijk and Stokhof, 1991, van Eijck and Visser, 1996]. Prism embeds this idea at the level of a very small, concrete notation.

8.2 Lexicon vs. Grammar

Contexts in Prism play a role similar to that of lexicons in generative grammar [Chomsky, 1995]. The core (**Core1**) provides the abstract combinatorial machinery and a few very general categories. A context such as **ThermostatControl** introduces:

- new categories (`Location`, `Temperature`, `Action`),
- named constants (`office`, `celsius`),
- domain-specific predicates (`gtTemp`, `ltTemp`),
- and external actions (`lowerThermostat`, `raiseThermostat`, `maintainThermostat`).

The same core could be reused for a completely different domain (for example, cybersecurity incident response) by introducing new contexts with categories such as `System`, `Incident`, `Recommendation`, and so on, but with the same underlying boolean and tool machinery. This modularity reflects the separation between universal grammar and domain-specific lexical items in linguistic theory [Chomsky, 1995].

8.3 Tools as Bridges Between Language and World

The `Tool` structure in `Core1` is intentionally generic. It can describe capabilities exposed via external protocols such as web APIs or MCP tools [Anthropic, 2024], but Prism itself does not prescribe how the underlying communication works.

From a linguistic viewpoint, `Tool` can be understood as a bridge between the agent’s internal policy language and the external world. Role categories such as `Action` or `Response` refine this general notion; they behave like semantic roles or verb classes. A function that expects an `Action` is implicitly a function that anticipates something executable, but the fine-grained distinction between different roles prevents arbitrary re-use.

9 Why a Language Like Prism is Useful

There are several reasons to consider a small metalanguage like Prism for agents:

- **Inspectability.** Because agent policies are expressed as compact, compositional expressions, they can be inspected and reasoned about without delving into model internals.
- **Control and safety.** The set of allowed actions and tools is explicitly enumerated in contexts. This makes it easier to state, and check, safety constraints such as “the agent may not call these tools under these conditions”.
- **Separation of concerns.** Linguistically rich prompts and context can still be handled by large language models, but the final decision over *what to do* is constrained by a Prism policy. This suggests a hybrid architecture where natural language understanding and formal control coexist.
- **Reusability.** The same `Core1` can underlie very different domains; contexts serve as small, reusable grammars of agent behaviour for specific applications.
- **Verifiability.** The formal structure allows for automated verification of properties such as “no critical action is taken without proper authorization” or “all sensor readings are validated before use”.

For linguists, Prism can be read as a proposal for a very small interface language between semantic/pragmatic representations and computational agents. It makes explicit the structure of decisions and the vocabulary of tools, while staying agnostic about the details of underlying algorithms.

10 Conclusion

Prism is a minimalist metalanguage for agent behaviour. It is built out of a tiny core of abstract combinators (booleans, predicates, pairs, tools) and a notion of contexts that introduce domain-specific categories and external capabilities. Decisions that might usually be expressed with imperative control flow are here written as ordinary expressions that select among alternatives.

This design is motivated by both linguistic and engineering concerns: we want agent behaviour to be compositional, lexically transparent, and formally inspectable, while still rich enough to describe real-world tasks. The examples in this paper demonstrate how Prism can specify diverse agent behaviours—from thermostat control to medical monitoring—using the same underlying compositional machinery.

Future work could connect Prism more directly to existing frameworks in formal semantics and dialogue modelling, and explore how linguists might use Prism-like languages to specify the behaviour of tool-using agents in a principled way. Additionally, developing tooling for automated verification and policy synthesis would make Prism more practical for deployment in safety-critical applications.

References

- Anthropic. Model Context Protocol. <https://www.anthropic.com/news/model-context-protocol>, 2024.
- Noam Chomsky. *The Minimalist Program*. MIT Press, Cambridge, MA, 1995.
- Gottlob Frege. Über Sinn und Bedeutung. *Zeitschrift für Philosophie und philosophische Kritik*, 100:25–50, 1892.
- Jeroen Groenendijk and Martin Stokhof. Dynamic predicate logic. *Linguistics and Philosophy*, 14(1):39–100, 1991.
- Barbara H. Partee. Lexical semantics and compositionality. In Lila R. Gleitman and Mark Liberman, editors, *An Invitation to Cognitive Science, Vol. 1: Language*, pages 311–360. MIT Press, Cambridge, MA, 1995.
- Paul Taylor. *Practical Foundations of Mathematics*. Cambridge University Press, Cambridge, 1999. Available at <https://www.paultaylor.eu/stable/prot.pdf>.
- Jan van Eijck and Albert Visser. Dynamic semantics. In Johan van Benthem and Alice ter Meulen, editors, *Handbook of Logic and Language*, pages 587–648. Elsevier, Amsterdam, 1996.